S.Harini

192425171

Experiment 1: Breadth First Search (BFS)

Aim: To find the shortest path in an unweighted graph by exploring nodes level by level.

Algorithm:

1. Start from the source node and mark it as visited.
2. Insert the source node into a queue.
3. Remove the front node from the queue and check its adjacent nodes.
4. Add each unvisited adjacent node to the queue and mark it as visited.
5. Repeat until the destination is found or the queue becomes empty.
6. Reconstruct the path using the stored parent nodes, if required.

Program & Output:

EX-AI-1.py - C:/Users/user/Downloads/EX-AI-1.py (3.13.1)

File Edit Format Run Options Window Help

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = set()

    print("BREADTH FIRST SEARCH")
    print("Starting Node:", start)
    print("Goal Node:", goal)

    while queue:
        node, path = queue.popleft()

        if node in visited:
            continue

        visited.add(node)
        print("Visiting Node:", node)
        print("Current Path:", path)

        if node == goal:
            print("Goal Reached!")
            print("Shortest Path:", " -> ".join(path))
            print("Total Steps:", len(path) - 1)
            print("Visited Nodes:", visited)
            return

        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))
```

Ln: 48 Col: 0

IDLE Shell 3.13.1

File Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/EX-AI-1.py ==========
========
BREADTH FIRST SEARCH
Starting Node: A
Goal Node: G
Visiting Node: A
Current Path: ['A']
Visiting Node: B
Current Path: ['A', 'B']
Visiting Node: C
Current Path: ['A', 'C']
Visiting Node: D
Current Path: ['A', 'B', 'D']
Visiting Node: E
Current Path: ['A', 'B', 'E']
Visiting Node: F
Current Path: ['A', 'C', 'F']
Visiting Node: G
Current Path: ['A', 'C', 'G']
Goal Reached!
Shortest Path: A -> C -> G
Total Steps: 2
Visited Nodes: {'B', 'E', 'C', 'D', 'A', 'G', 'F'}
>>>
```

Ln: 26 Col: 0

Result: The BFS algorithm explores nodes level-wise and finds the shortest path in an unweighted graph. For example, the path from A to D may be A → B → D.

Experiment 2: Depth First Search (DFS)

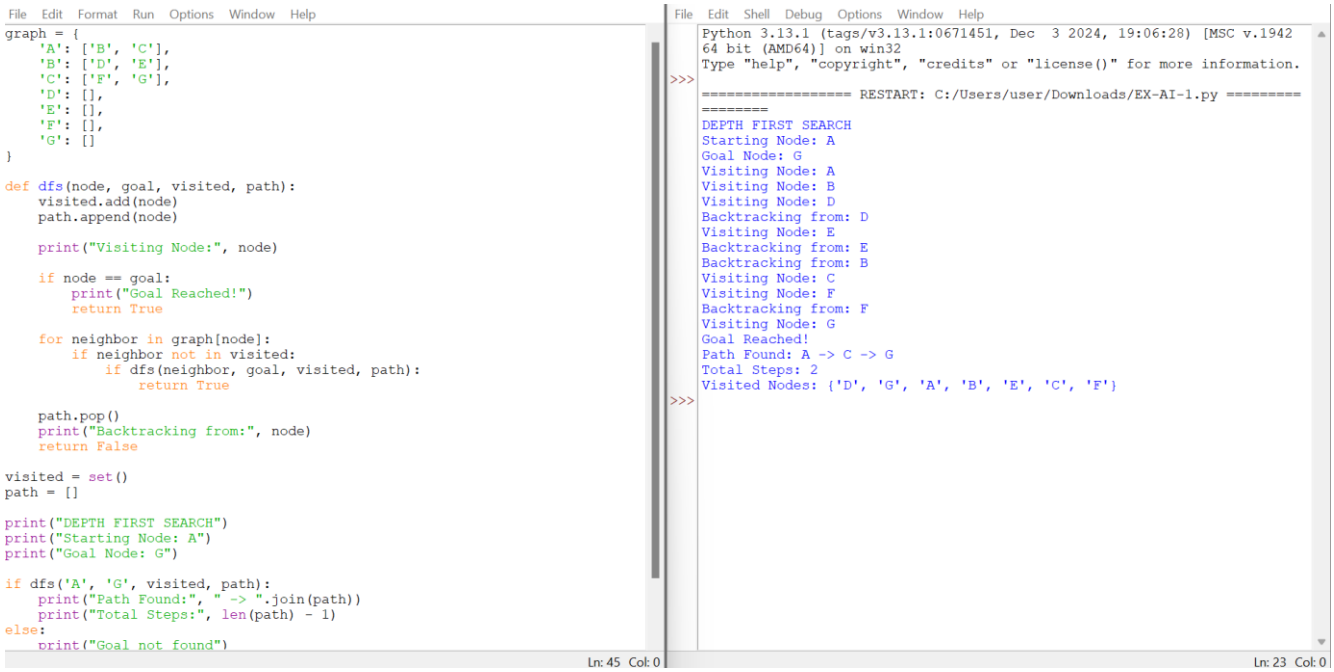
Aim: To traverse a graph by exploring one branch as deeply as possible before backtracking.

Algorithm:

1. Start at the source node and mark it as visited.
2. Visit an unvisited adjacent node.
3. Continue moving to an unvisited adjacent node until no further move is possible.
4. Backtrack to the previous node and explore another unvisited branch.

5. Repeat until all reachable nodes have been visited

Program & Output:



```
File  Edit  Format  Run  Options  Window  Help
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def dfs(node, goal, visited, path):
    visited.add(node)
    path.append(node)

    print("Visiting Node:", node)

    if node == goal:
        print("Goal Reached!")
        return True

    for neighbor in graph[node]:
        if neighbor not in visited:
            if dfs(neighbor, goal, visited, path):
                return True

    path.pop()
    print("Backtracking from:", node)
    return False

visited = set()
path = []

print("DEPTH FIRST SEARCH")
print("Starting Node: A")
print("Goal Node: G")

if dfs('A', 'G', visited, path):
    print("Path Found:", " -> ".join(path))
    print("Total Steps:", len(path) - 1)
else:
    print("Goal not found")
```

Ln: 45 Col: 0

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/EX-AI-1.py =========
========
DEPTH FIRST SEARCH
Starting Node: A
Goal Node: G
Visiting Node: A
Visiting Node: B
Visiting Node: D
Backtracking from: D
Visiting Node: E
Backtracking from: E
Backtracking from: B
Visiting Node: C
Visiting Node: F
Backtracking from: F
Visiting Node: G
Goal Reached!
Path Found: A -> C -> G
Total Steps: 2
Visited Nodes: {'D', 'G', 'A', 'B', 'E', 'C', 'F'}
>>>
```

Ln: 23 Col: 0

Result: DFS visits nodes deeply before backtracking and produces a depth-first traversal of the graph.

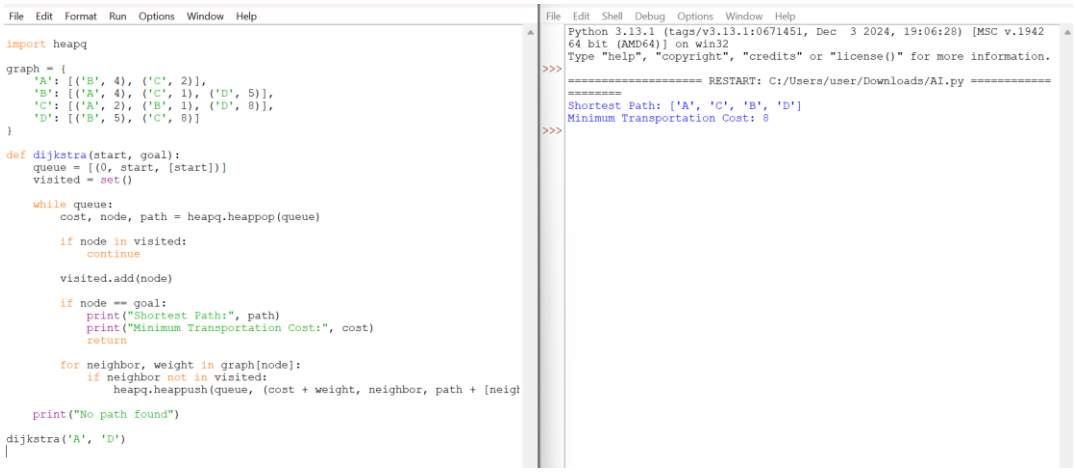
Experiment 3: Uniform Cost Search (UCS)

Aim: To find the least-cost path from a source node to a destination in a weighted graph.

Algorithm:

7. Insert the source node into a priority queue with cost 0.
8. Remove the node with the lowest cumulative path cost.
9. If it is the goal, return the path and its total cost.
10. Otherwise, expand its unvisited or lower-cost neighbours.
11. Insert or update each neighbour with the new cumulative cost.
12. Repeat until the goal is removed from the queue.

Program & Output:



```
import heapq

graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('C', 1), ('D', 5)],
    'C': [('A', 2), ('B', 1), ('D', 8)],
    'D': [('B', 5), ('C', 8)]
}

def dijkstra(start, goal):
    queue = [(0, start, [start])]
    visited = set()

    while queue:
        cost, node, path = heapq.heappop(queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Shortest Path:", path)
            print("Minimum Transportation Cost:", cost)
            return

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(queue, (cost + weight, neighbor, path + [neigh

    print("No path found")

dijkstra('A', 'D')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AI.py ==========
========
Shortest Path: ['A', 'C', 'B', 'D']
Minimum Transportation Cost: 8
>>>
```

Result: UCS returns the path with the minimum total transportation cost, provided edge costs are non-negative.

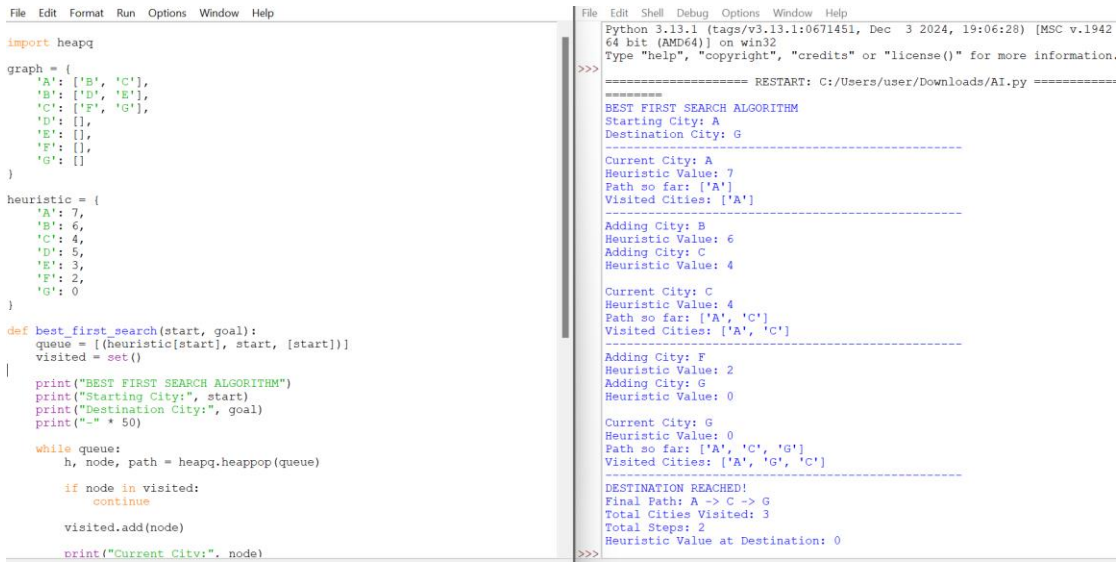
Experiment 4: Best First Search

Aim: To find a path by selecting the node that appears closest to the goal according to a heuristic value.

Algorithm:

1. Place the start node in a priority queue.
2. Assign each node a heuristic value estimating its distance to the goal.
3. Select the node with the most promising heuristic value.
4. If the selected node is the goal, return the path.
5. Otherwise, add its unvisited neighbouring nodes to the priority queue.
6. Continue until the goal is found or no nodes remain.

Program & Output:



```
import heapq

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 5,
    'E': 3,
    'F': 2,
    'G': 0
}

def best_first_search(start, goal):
    queue = [(heuristic[start], start, [start])]
    visited = set()

    print("BEST FIRST SEARCH ALGORITHM")
    print("Starting City:", start)
    print("Destination City:", goal)
    print("-" * 50)

    while queue:
        h, node, path = heapq.heappop(queue)

        if node in visited:
            continue

        visited.add(node)

        print("Current City:", node)
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AI.py ==========
========
BEST FIRST SEARCH ALGORITHM
Starting City: A
Destination City: G
--------------------------------------------------
Current City: A
Heuristic Value: 7
Path so far: ['A']
Visited Cities: ['A']
--------------------------------------------------
Adding City: B
Heuristic Value: 6
Adding City: C
Heuristic Value: 4

Current City: C
Heuristic Value: 4
Path so far: ['A', 'C']
Visited Cities: ['A', 'C']
--------------------------------------------------
Adding City: F
Heuristic Value: 2
Adding City: G
Heuristic Value: 0

Current City: G
Heuristic Value: 0
Path so far: ['A', 'C', 'G']
Visited Cities: ['A', 'G', 'C']
--------------------------------------------------
DESTINATION REACHED!
Final Path: A -> C -> G
Total Cities Visited: 3
Total Steps: 2
Heuristic Value at Destination: 0
>>>
```

Result: Best First Search selects nodes according to heuristic evaluation and returns a path to the destination when one is found.

Experiment 5: Greedy Best First Search

Aim: To solve a pathfinding problem by choosing the next node with the smallest estimated distance to the goal.

Algorithm:

1. Start with the initial node and place it in a priority queue.
2. Evaluate neighbouring nodes using the heuristic h(n).
3. Choose the node with the lowest heuristic value.
4. Mark the chosen node as visited and record its parent.
5. Add its unvisited neighbours to the priority queue.
6. Continue until the destination is reached, then reconstruct the path.

Program & Output:

```
import heapq

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

h = {'A': 7, 'B': 6, 'C': 4, 'D': 5, 'E': 3, 'F': 2, 'G': 0}

def greedy_search(start, goal):
    queue = [(h[start], start, [start])]
    visited = set()

    while queue:
        cost, node, path = heapq.heappop(queue)

        if node in visited:
            continue

        visited.add(node)
        print("Visiting:", node, "Heuristic:", h[node])

        if node == goal:
            print("Final Path:", " -> ".join(path))
            print("Total Steps:", len(path) - 1)
            return

        for neighbor in graph[node]:
            if neighbor not in visited:
                heapq.heappush(queue, (h[neighbor], neighbor, path + [neighbo

    print("Path not found")

greedy_search('A', 'G')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
Visiting: A Heuristic: 7
Visiting: C Heuristic: 4
Visiting: G Heuristic: 0
Final Path: A -> C -> G
Total Steps: 2
>>>
```

Result: The algorithm finds a heuristic-guided path by choosing the apparently nearest node. It is not guaranteed to find the minimum-cost path.

Experiment 6: A* Search

Aim: To find an optimal path by combining the cost already travelled and the estimated remaining cost.

Algorithm:

1. Insert the start node with g(n) = 0 and calculate f(n) = g(n) + h(n).
2. Select the node with the lowest f-value.
3. If the selected node is the goal, reconstruct and return the path.
4. Expand its neighbours and calculate their new path costs g(n).
5. Update a neighbour when a lower path cost is found.
6. Repeat until the goal is reached or the open list is empty.

Program & Output:

```
import heapq

raph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []

 = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

ef astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best = {start: 0}

    while queue:
        f, g, node, path = heapq.heappop(queue)

        if g != best.get(node):
            continue

        print("Node:", node, "g =", g, "h =", h[node], "f =", f)

        if node == goal:
            print("Optimal Path:", " -> ".join(path))
            print("Minimum Cost:", g)
            return

        for neighbor, cost in graph[node]:
            new_g = g + cost

            if new_g < best.get(neighbor, float('inf')):
                best[neighbor] = new_g
                heapq.heappush(queue, (new_g + h[neighbor],
                                       new_g, neighbor, path + [neighbor]))

star('A', 'D')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for m
>>>
========== RESTART: C:/Users/user/Downloads/AIE
========
Node: A g = 0 h = 4 f = 4
Node: B g = 1 h = 3 f = 4
Node: C g = 3 h = 1 f = 4
Node: D g = 4 h = 0 f = 4
Optimal Path: A -> B -> C -> D
Minimum Cost: 4
>>>
```

Result: A* finds an optimal path when the heuristic is admissible and the edge costs are non-negative. The evaluation function is f(n) = g(n) + h(n).

Experiment 7: Minimax Algorithm

Aim: To select the best move in a two-player, turn-based game.

Algorithm:

1. Construct the game tree from the current game state.
2. Generate possible moves until a terminal state or depth limit is reached.
3. Assign utility values to the leaf nodes.
4. At MAX levels, choose the maximum child value.
5. At MIN levels, choose the minimum child value.
6. Select the move with the best value for the maximizing player.

Program & Output:

```
def minimax(depth, node, maximizing, values):
    if depth == 2:
        return values[node]

    if maximizing:
        return max(
            minimax(depth + 1, node * 2, False, values),
            minimax(depth + 1, node * 2 + 1, False, values)
        )
    else:
        return min(
            minimax(depth + 1, node * 2, True, values),
            minimax(depth + 1, node * 2 + 1, True, values)
        )

values = [3, 5, 2, 9]
print("Leaf Scores:", values)
print("Best Score:", minimax(0, 0, True, values))
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
Leaf Scores: [3, 5, 2, 9]
Best Score: 3
>>>
```

Result: Minimax selects the move that maximizes the player's guaranteed outcome, assuming the opponent also plays optimally.

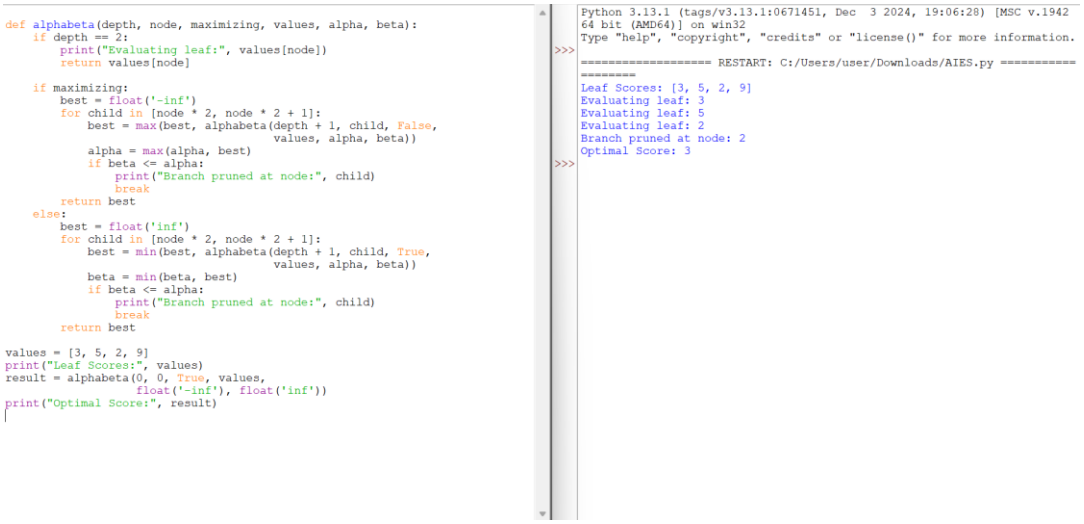
Experiment 8: Alpha-Beta Pruning

Aim: To reduce the number of nodes evaluated in Minimax without changing the final decision.

Algorithm:

1. Start the game-tree search with alpha = −∞ and beta = +∞.
2. At a MAX node, update alpha with the largest value found.
3. At a MIN node, update beta with the smallest value found.
4. If alpha is greater than or equal to beta, stop exploring the remaining children of that node.
5. Continue the search for the remaining branches.
6. Choose the move with the resulting Minimax value.

Program & Output:



```
def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        print("Evaluating leaf:", values[node])
        return values[node]

    if maximizing:
        best = float('-inf')
        for child in [node * 2, node * 2 + 1]:
            best = max(best, alphabeta(depth + 1, child, False,
                                       values, alpha, beta))
            alpha = max(alpha, best)
            if beta <= alpha:
                print("Branch pruned at node:", child)
                break
        return best
    else:
        best = float('inf')
        for child in [node * 2, node * 2 + 1]:
            best = min(best, alphabeta(depth + 1, child, True,
                                       values, alpha, beta))
            beta = min(beta, best)
            if beta <= alpha:
                print("Branch pruned at node:", child)
                break
        return best

values = [3, 5, 2, 9]
print("Leaf Scores:", values)
result = alphabeta(0, 0, True, values,
                   float('-inf'), float('inf'))
print("Optimal Score:", result)
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
Leaf Scores: [3, 5, 2, 9]
Evaluating leaf: 3
Evaluating leaf: 5
Evaluating leaf: 2
Branch pruned at node: 2
Optimal Score: 3
>>>
```

Result: Alpha-Beta Pruning eliminates branches that cannot affect the final decision and produces the same optimal move as Minimax.

Experiment 9: Water Jug Problem

Aim: To measure an exact quantity of water using two containers of fixed capacities.

Algorithm:

1. Represent the state as (x, y), where x and y are the current water quantities.
2. Start with both jugs empty: (0, 0).
3. Generate valid operations: fill a jug, empty a jug, or pour water from one jug to the other.
4. Apply an operation to obtain a new state.
5. Do not revisit states that have already been explored.
6. Continue until either jug contains the required quantity.

Program & Output:

```python
from collections import deque

def water_jug(cap1, cap2, target):
    queue = deque([(0, 0, [])])
    visited = set()

    while queue:
        a, b, path = queue.popleft()

        if (a, b) in visited:
            continue

        visited.add((a, b))
        path = path + [(a, b)]

        if a == target or b == target:
            for step in path:
                print(step)
            print("Target reached:", target, "litres")
            return

        states = [
            (cap1, b),
            (a, cap2),
            (0, b),
            (a, 0),
            (a - min(a, cap2 - b), b + min(a, cap2 - b)),
            (a + min(b, cap1 - a), b - min(b, cap1 - a))
        ]

        for state in states:
            if state not in visited:
                queue.append((state[0], state[1], path))

water_jug(4, 3, 2)
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
Target reached: 2 litres
>>>
```

Result: The sequence of filling, emptying and pouring operations reaches the required water quantity. The solution depends on the jug capacities and target amount.

Experiment 10: Towers of Hanoi

Aim: To move a stack of disks from the source peg to the destination peg using an auxiliary peg.

Algorithm:

1. If only one disk remains, move it directly to the destination peg.
2. Move the top n−1 disks from the source peg to the auxiliary peg.
3. Move the largest disk from the source peg to the destination peg.
4. Move the n−1 disks from the auxiliary peg to the destination peg.
5. Repeat the recursive process until all disks are transferred.

Program & Output:

```python
def hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    hanoi(n - 1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    hanoi(n - 1, auxiliary, source, destination)

n = 3
print("TOWERS OF HANOI")
print("Number of disks:", n)
hanoi(n, 'A', 'B', 'C')
print("Total moves:", 2**n - 1)
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
TOWERS OF HANOI
Number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
>>>
```

Result: All disks are transferred while following the rule that a larger disk must never be placed on a smaller disk. The minimum number of moves is $2^n - 1$.

Experiment 11: Crypt Arithmetic Problem

Aim: To assign a unique digit to each letter so that a given arithmetic equation is mathematically correct.

Algorithm:

1. Identify the distinct letters in the arithmetic expression.
2. Assign a different digit to each letter.
3. Ensure that leading letters are not assigned zero.
4. Substitute the assigned digits into the equation.
5. Check whether the arithmetic operation is correct.
6. Accept assignments that satisfy all constraints.

Program & Output:

```
from itertools import permutations

letters = 'SENDMORY'
digits = '0123456789'

print("CRYPT ARITHMETIC: SEND + MORE = MONEY")

for p in permutations(digits, len(letters)):
    d = dict(zip(letters, p))

    if d['S'] == '0' or d['M'] == '0':
        continue

    send = int(''.join(d[x] for x in 'SEND'))
    more = int(''.join(d[x] for x in 'MORE'))
    money = int(''.join(d[x] for x in 'MONEY'))

    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        print("Result:", send, "+", more, "=", money)
        break
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
CRYPT ARITHMETIC: SEND + MORE = MONEY
SEND = 9567
MORE = 1085
MONEY = 10652
Result: 9567 + 1085 = 10652
>>>
```

Result: A valid digit assignment is obtained when every letter has a unique digit and the arithmetic equation is satisfied. For SEND + MORE = MONEY, one solution is 9567 + 1085 = 10652.

Experiment 12: Decision Tree Classifier

Aim: To classify loan applications as approved or rejected using customer information.

Algorithm:

1. Collect and prepare the customer dataset with features and class labels.
2. Separate the dataset into training and testing sets.
3. Train a Decision Tree Classifier using the training data.
4. The model selects features and split points that best separate the classes.
5. Predict the class labels for the test data.
6. Evaluate performance using accuracy and a confusion matrix.

Program & Output:

```
from sklearn.tree import DecisionTreeClassifier

X = [
    [25, 20000],
    [30, 25000],
    [45, 60000],
    [50, 70000],
    [28, 22000],
    [40, 55000]
]

y = ['Reject', 'Reject', 'Approve', 'Approve',
     'Reject', 'Approve']

model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

print("LOAN APPROVAL SYSTEM")
print("Training samples:", len(X))

tests = [[35, 40000], [24, 18000], [48, 65000]]

for person in tests:
    result = model.predict([person])[0]
    print("Age:", person[0],
          "Income:", person[1],
          "Decision:", result)
```

```
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========

========== RESTART: C:/Users/user/Downloads/AIES.py ==========
LOAN APPROVAL SYSTEM
Training samples: 6
Age: 35 Income: 40000 Decision: Reject
Age: 24 Income: 18000 Decision: Reject
Age: 48 Income: 65000 Decision: Approve
>>>
```

Result: The trained decision tree predicts whether a loan application should be approved or rejected based on the input customer details.

Experiment 13: Artificial Neural Network

Aim: To train a neural network to predict whether a patient has a disease using medical records.

Algorithm:

1. Collect and clean the medical dataset.
2. Separate input features and target labels.
3. Split the data into training and testing sets and scale features when required.
4. Create a neural network with input, hidden and output layers.
5. Train the network using a suitable loss function and optimizer.
6. Test the trained model and calculate evaluation metrics.

Program & Output:

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = [
    [1, 20], [2, 25], [3, 30], [4, 35],
    [5, 45], [6, 50], [7, 60], [8, 65],
    [2, 22], [3, 28], [6, 55], [7, 62]
]

y = [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(5,), max_iter=3000,
                  random_state=1)
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("ARTIFICIAL NEURAL NETWORK")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Actual values:", list(y_test))
print("Predicted values:", list(predictions))
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
ARTIFICIAL NEURAL NETWORK
Training samples: 9
Testing samples: 3
Actual values: [1, 0, 1]
Predicted values: [np.int64(1), np.int64(0), np.int64(1)]
Accuracy: 100.0 %
>>>
```

Result: The neural network learns patterns from historical medical records and predicts the disease class for test patients. Its performance is measured using suitable metrics such as accuracy, precision and recall.

Experiment 14: Diet Recommendation Expert System

Aim: To recommend a suitable diet plan using rules based on age, BMI and health conditions.

Algorithm:

1. Read the person's age, height, weight and health condition.
2. Calculate BMI using weight (kg) / height² (m²).
3. Apply predefined rules to interpret BMI and health information.
4. Select suitable food recommendations based on the rules.
5. Display a diet suggestion with appropriate meal choices.
6. Review the recommendation with a qualified nutrition professional when needed.

Program & Output:

```
def recommend_diet(age, bmi, condition):
    print("DIET RECOMMENDATION SYSTEM")
    print("Age:", age)
    print("BMI:", bmi)
    print("Condition:", condition)

    if condition == "diabetes":
        diet = "Balanced high-fibre meals with controlled carbohydrates"
    elif condition == "pcos":
        diet = "Balanced meals with fibre, protein and whole grains"
    elif bmi >= 25:
        diet = "Balanced meals with vegetables, protein and whole grains"
    elif bmi < 18.5:
        diet = "Nutrient-dense meals with adequate calories and protein"
    else:
        diet = "Balanced diet with fruits, vegetables and protein"

    print("Recommended Diet:", diet)
    print("Suggestions:")
    print("- Include vegetables and whole grains")
    print("- Choose suitable protein sources")
    print("- Drink enough water")

recommend_diet(25, 27.5, "pcos")
```

```
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
DIET RECOMMENDATION SYSTEM
Age: 25
BMI: 27.5
Condition: pcos
Recommended Diet: Balanced meals with fibre, protein and whole grains
Suggestions:
- Include vegetables and whole grains
- Choose suitable protein sources
- Drink enough water
>>>
```

Result: The expert system produces a rule-based diet recommendation based on the entered details. It provides general guidance and does not replace professional medical or nutrition advice.

Experiment 15: Family Knowledge Representation

Aim: To represent family facts and infer relationships such as parent, sibling and grandparent.

Algorithm:

1. Store basic facts such as parent(person1, person2).
2. Define rules for relationships using the stored facts.
3. Define a sibling rule: two people are siblings if they share a parent and are different people.
4. Define a grandparent rule using two parent relationships.
5. Submit queries to the knowledge base.
6. Display the relationships that satisfy the rules.

Program & Output:

```python
parents = {
    'Ravi': ['Arun', 'Priya'],
    'Meena': ['Arun', 'Priya'],
    'Arun': ['Kiran']
}

gender = {
    'Ravi': 'male',
    'Meena': 'female',
    'Arun': 'male',
    'Priya': 'female',
    'Kiran': 'male'
}

def find_parent(child):
    for parent, children in parents.items():
        if child in children:
            print(parent, "is the parent of", child)

def find_siblings(person):
    siblings = set()
    for parent, children in parents.items():
        if person in children:
            siblings.update(children)
    siblings.discard(person)
    print("Siblings of", person, ":", sorted(siblings))

def find_grandparents(person):
    for parent, children in parents.items():
        if person in children:
            find_parent(parent)

print("FAMILY KNOWLEDGE BASE")
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
FAMILY KNOWLEDGE BASE
Ravi is the parent of Arun
Meena is the parent of Arun
Siblings of Arun : ['Priya']
Arun is the parent of Kiran
>>>
```

Result: The knowledge base infers family relationships from stored facts. For example, it can identify a grandparent through two parent links.

Experiment 16: Birds Fly Knowledge Base

Aim: To represent bird characteristics and infer whether a bird can fly using inheritance-based rules.

Algorithm:

1. Store general facts about birds.
2. Define a general rule that birds can fly.
3. Store exceptions for birds that cannot fly, such as penguins and ostriches.
4. Check whether the queried animal is a bird.
5. Apply the exception rule before the general flying rule.
6. Return whether the bird can fly or cannot fly.

Program & Output:

```python
birds = {
    'Sparrow': {'can_fly': True},
    'Parrot': {'can_fly': True},
    'Eagle': {'can_fly': True},
    'Penguin': {'can_fly': False},
    'Ostrich': {'can_fly': False}
}

def check_flying(bird):
    if bird in birds:
        if birds[bird]['can_fly']:
            print(bird, "is a bird and can fly")
        else:
            print(bird, "is a bird but cannot fly")
    else:
        print("Bird not found")

print("BIRDS FLY KNOWLEDGE BASE")
check_flying('Sparrow')
check_flying('Eagle')
check_flying('Penguin')
check_flying('Ostrich')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
BIRDS FLY KNOWLEDGE BASE
Sparrow is a bird and can fly
Eagle is a bird and can fly
Penguin is a bird but cannot fly
Ostrich is a bird but cannot fly
>>>
```

Result: The knowledge base uses inheritance to infer that most birds can fly while handling exceptions such as penguins and ostriches.

Experiment 17: Medical Expert System

Aim: To identify possible diseases from patient symptoms using predefined rules.

Algorithm:

1. Accept symptoms entered by the user.
2. Store medical rules that connect symptom combinations to possible conditions.
3. Compare the entered symptoms with the rule conditions.
4. Fire the rules whose conditions are satisfied.
5. Display the possible disease or recommendation for further evaluation.
6. Explain that the result is preliminary and requires clinical confirmation.

Program & Output:

```
def diagnose(symptoms):
    print("MEDICAL EXPERT SYSTEM")
    print("Symptoms:", symptoms)

    if 'fever' in symptoms and 'cough' in symptoms:
        print("Possible condition: Respiratory infection")
    elif 'sneezing' in symptoms and 'runny nose' in symptoms:
        print("Possible condition: Common cold or allergy")
    elif 'headache' in symptoms and 'nausea' in symptoms:
        print("Possible condition: Migraine")
    elif 'fever' in symptoms:
        print("Possible condition: Fever requiring assessment")
    else:
        print("Insufficient information for a rule-based result")

    print("Consult a qualified healthcare professional for diagnosis")

diagnose(['fever', 'cough'])
diagnose(['headache', 'nausea'])
```

```
Python 3.13.1 (tags/v3.13.1:06714517, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
MEDICAL EXPERT SYSTEM
Symptoms: ['fever', 'cough']
Possible condition: Respiratory infection
Consult a qualified healthcare professional for diagnosis
MEDICAL EXPERT SYSTEM
Symptoms: ['headache', 'nausea']
Possible condition: Migraine
Consult a qualified healthcare professional for diagnosis
>>>
```

Result: The expert system infers possible conditions from the supplied symptoms. It provides preliminary information and is not a substitute for diagnosis by a healthcare professional.

Experiment 18: Fruit Color Classification

Aim: To classify fruits based on their observed colors using production rules.

Algorithm:

1. Read the color of the fruit as input.
2. Store production rules that map colors to possible fruit classes.
3. Compare the input color with the rules.
4. Select the fruit category whose rule matches.
5. Display the classification result.
6. If multiple fruits share the same color, request additional features such as shape or size.

Program & Output:

```python
def classify_fruit(color):
    rules = {
        'red': 'Apple or Cherry',
        'yellow': 'Banana or Lemon',
        'orange': 'Orange',
        'green': 'Green Apple or Guava',
        'purple': 'Grapes'
    }

    print("Fruit Color Classification")
    print("Input Color:", color)

    if color.lower() in rules:
        print("Possible Fruit:", rules[color.lower()])
    else:
        print("Fruit color not recognized")

classify_fruit('red')
classify_fruit('yellow')
classify_fruit('orange')
classify_fruit('green')
classify_fruit('purple')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
Fruit Color Classification
Input Color: red
Possible Fruit: Apple or Cherry
Fruit Color Classification
Input Color: yellow
Possible Fruit: Banana or Lemon
Fruit Color Classification
Input Color: orange
Possible Fruit: Orange
Fruit Color Classification
Input Color: green
Possible Fruit: Green Apple or Guava
Fruit Color Classification
Input Color: purple
Possible Fruit: Grapes
>>>
```

Result: The rule-based system classifies a fruit using its color. For example, yellow may suggest banana and orange may suggest orange, although color alone may not uniquely identify a fruit.

Experiment 19: Planet Knowledge Representation

Aim: To store facts about planets and answer queries using logical rules.

Algorithm:

1. Store facts such as planet name, order, size and other properties.
2. Define rules to derive additional information from the stored facts.
3. Accept a query about a planet.
4. Search the knowledge base for matching facts or rule conclusions.
5. Return the requested planetary information.

Program & Output:

```python
planets = {
    'Mercury': {'order': 1, 'type': 'Terrestrial'},
    'Venus': {'order': 2, 'type': 'Terrestrial'},
    'Earth': {'order': 3, 'type': 'Terrestrial'},
    'Mars': {'order': 4, 'type': 'Terrestrial'},
    'Jupiter': {'order': 5, 'type': 'Gas Giant'},
    'Saturn': {'order': 6, 'type': 'Gas Giant'},
    'Uranus': {'order': 7, 'type': 'Ice Giant'},
    'Neptune': {'order': 8, 'type': 'Ice Giant'}
}

def planet_info(name):
    if name in planets:
        print("Planet:", name)
        print("Order from Sun:", planets[name]['order'])
        print("Planet Type:", planets[name]['type'])
    else:
        print("Planet not found")

print("PLANET KNOWLEDGE BASE")
planet_info('Earth')
planet_info('Jupiter')
planet_info('Neptune')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
PLANET KNOWLEDGE BASE
Planet: Earth
Order from Sun: 3
Planet Type: Terrestrial
Planet: Jupiter
Order from Sun: 5
Planet Type: Gas Giant
Planet: Neptune
Order from Sun: 8
Planet Type: Ice Giant
>>>
```

Result: The knowledge base retrieves stored planetary facts and derives answers using logical rules.

Experiment 20: Person DOB Knowledge Base

Aim: To store people's dates of birth and retrieve details through logical queries.

Algorithm:

1. Store each person's name and date of birth as facts.
2. Define rules or predicates to query a person's birth details.
3. Accept a person's name as the query input.
4. Search the knowledge base for the matching record.
5. Display the date of birth if a matching fact exists; otherwise report no record.

Program & Output:

```
people = {
    'Arun': {'dob': '2004-05-12', 'city': 'Chennai'},
    'Priya': {'dob': '2003-08-20', 'city': 'Bengaluru'},
    'Rahul': {'dob': '2005-01-15', 'city': 'Hyderabad'},
    'Meena': {'dob': '2002-11-25', 'city': 'Chennai'}
}

def search_person(name):
    print("Searching for:", name)

    if name in people:
        print("Name:", name)
        print("Date of Birth:", people[name]['dob'])
        print("City:", people[name]['city'])
    else:
        print("Person not found")

print("PERSON DOB KNOWLEDGE BASE")
search_person('Arun')
search_person('Meena')
search_person('John')
```

```
Python 3.13.1 (tags/v3.13.1:06714517, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
PERSON DOB KNOWLEDGE BASE
Searching for: Arun
Name: Arun
Date of Birth: 2004-05-12
City: Chennai
Searching for: Meena
Name: Meena
Date of Birth: 2002-11-25
City: Chennai
Searching for: John
Person not found
>>>
```

Result: The system retrieves the date of birth associated with the queried person from the stored facts.

Experiment 21: Pattern Recognition

Aim: To identify recurring patterns in data using an AI or machine-learning technique.

Algorithm:

1. Collect the input data or images.
2. Preprocess the data by resizing, cleaning or extracting relevant features.
3. Choose a pattern-recognition method such as clustering or classification.
4. Train or fit the model using the prepared data.
5. Apply the model to identify patterns in new or existing data.
6. Evaluate the output using suitable measures.

Program & Output:

```python
from sklearn.cluster import KMeans
import numpy as np

data = np.array([
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 9],
    [9, 8],
    [10, 9]
])

model = KMeans(n_clusters=2, random_state=0, n_init=10)
model.fit(data)

print("PATTERN RECOGNITION")
print("---------------------------")
print("Input Data:")
print(data)

print("\nDetected Patterns:")
for i, point in enumerate(data):
    print(point, "belongs to Cluster", model.labels_[i])

print("\nCluster Centers:")
print(model.cluster_centers_)
print("\nNumber of Patterns Detected:", len(set(model.labels_)))
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
PATTERN RECOGNITION
---------------------------
Input Data:
[[ 1  2]
 [ 2  3]
 [ 3  2]
 [ 8  9]
 [ 9  8]
 [10  9]]

Detected Patterns:
[1 2] belongs to Cluster 1
[2 3] belongs to Cluster 1
[3 2] belongs to Cluster 1
[8 9] belongs to Cluster 0
[9 8] belongs to Cluster 0
[10  9] belongs to Cluster 0

Cluster Centers:
[[9.         8.66666667]
 [2.         2.33333333]]

Number of Patterns Detected: 2
>>>
```

Result: The pattern-recognition system groups or classifies data according to learned similarities. The result depends on the quality of the input data and selected method.

Experiment 22: Monkey and Banana Problem

Aim: To plan a sequence of actions that allows a monkey to obtain a banana placed out of reach.

Algorithm:

1. Represent the initial state, including the monkey, box and banana positions.
2. Identify the goal state in which the monkey has the banana.
3. Define actions such as walk, push box, climb box and grasp banana.
4. Check the conditions required for each action.
5. Choose a valid sequence of actions that changes the initial state into the goal state.
6. Stop when the monkey successfully grasps the banana.

Program & Output:

```
monkey = "door"
box = "window"
banana = "middle"
has_banana = False

print("MONKEY AND BANANA PROBLEM")
print("----------------------------")
print("Monkey position:", monkey)
print("Box position:", box)
print("Banana position:", banana)

print("\nStep 1: Monkey moves from door to window")
monkey = "window"
print("Monkey position:", monkey)

print("\nStep 2: Monkey pushes the box to the middle")
box = "middle"
monkey = "middle"
print("Monkey position:", monkey)
print("Box position:", box)

print("\nStep 3: Monkey climbs onto the box")
on_box = True
print("Monkey is on the box:", on_box)

print("\nStep 4: Monkey grabs the banana")
has_banana = True
print("Monkey has the banana:", has_banana)

if has_banana:
    print("\nGoal Achieved! Monkey gets the banana.")
else:
    print("\nGoal not achieved.")
```

Ln: 34 Col: 0

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> ========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
MONKEY AND BANANA PROBLEM
----------------------------
Monkey position: door
Box position: window
Banana position: middle

Step 1: Monkey moves from door to window
Monkey position: window

Step 2: Monkey pushes the box to the middle
Monkey position: middle
Box position: middle

Step 3: Monkey climbs onto the box
Monkey is on the box: True

Step 4: Monkey grabs the banana
Monkey has the banana: True

Goal Achieved! Monkey gets the banana.
>>>
```

Ln: 25 Col: 0

Result: A valid plan is: walk to the box, push the box below the banana, climb the box and grasp the banana.

Experiment 23: Forward Chaining

Aim: To derive new conclusions from known facts by repeatedly applying rules.

Algorithm:

1. Store the initial facts in a knowledge base.
2. Store rules in IF–THEN form.
3. Check which rules have conditions matching the current facts.
4. Fire a matching rule and add its conclusion as a new fact.
5. Repeat the process using the updated facts.
6. Stop when the required conclusion is derived or no new facts can be added.

Program & Output:

```
facts = {
    "computer_not_starting",
    "power_light_off"
}

rules = [
    ({"power_light_off"}, "check_power_supply"),
    ({"check_power_supply"}, "power_supply_fault"),
    ({"power_supply_fault"}, "computer_needs_repair")
]

print("FORWARD CHAINING")
print("----------------------------")
print("Initial Facts:", facts)

changed = True

while changed:
    changed = False

    for conditions, conclusion in rules:
        if conditions.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            print("Rule Applied:", conditions, "->", conclusion)
            print("New Fact:", conclusion)
            changed = True

print("\nFinal Knowledge Base:")
for fact in sorted(facts):
    print(fact)

print("\nConclusion: Computer needs repair.")
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> ========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
FORWARD CHAINING
----------------------------
Initial Facts: {'computer_not_starting', 'power_light_off'}
Rule Applied: {'power_light_off'} -> check_power_supply
New Fact: check_power_supply
Rule Applied: {'check_power_supply'} -> power_supply_fault
New Fact: power_supply_fault
Rule Applied: {'power_supply_fault'} -> computer_needs_repair
New Fact: computer_needs_repair

Final Knowledge Base:
check_power_supply
computer_needs_repair
computer_not_starting
power_light_off
power_supply_fault

Conclusion: Computer needs repair.
>>>
```

Result: Forward chaining derives conclusions from available facts. For example, if a computer has no power and the rule says no power implies a possible power-supply fault, the system derives that fault.

Experiment 24: Backward Chaining

Aim: To determine whether a goal is true by working backward through rules and facts.

Algorithm:

1. Choose the conclusion to be proved as the goal.
2. Check whether the goal is already present as a fact.
3. If not, find a rule whose conclusion matches the goal.
4. Treat the conditions of that rule as subgoals.
5. Try to prove each subgoal using facts or other rules.
6. If all subgoals are proved, the original goal succeeds; otherwise it fails.

Program & Output:

```
facts = {
    "fever",
    "cough"
}

rules = {
    "flu": ["fever", "cough"],
    "respiratory_infection": ["fever", "cough"],
    "cold": ["cough", "sneezing"]
}

def backward_chaining(goal):
    print("Checking:", goal)

    if goal in facts:
        print(goal, "is a known fact")
        return True

    if goal in rules:
        for condition in rules[goal]:
            if not backward_chaining(condition):
                print("Goal failed:", goal)
                return False

        print("Goal proved:", goal)
        return True

    print("No rule or fact found for:", goal)
    return False

print("BACKWARD CHAINING")
print("-----------------------------")

goal = "flu"
print("Goal:", goal)

if backward_chaining(goal):
    print("\nConclusion: Flu is supported by the rules.")
else:
    print("\nConclusion: Goal cannot be proved.")
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
BACKWARD CHAINING
-----------------------------
Goal: flu
Checking: flu
Checking: fever
fever is a known fact
Checking: cough
cough is a known fact
Goal proved: flu

Conclusion: Flu is supported by the rules.
>>>
```

Result: Backward chaining works from a goal toward supporting facts and determines whether the requested conclusion can be proved.

Experiment 25: Best First Search in Prolog

Aim: To implement heuristic-based pathfinding in Prolog and locate a route to a goal node.

Algorithm:

1. Represent the graph using edge facts.
2. Store heuristic values for the nodes.
3. Start the search with a path containing the initial node.
4. Generate paths to unvisited neighbouring nodes.
5. Sort the available paths according to the heuristic value of their leading node.
6. Select the most promising path and repeat until the goal is reached.
7. Reverse the stored path before displaying it, if paths are stored goal-to-start.

Program & Output:

Result: The Prolog program uses edge facts and heuristic values to find a route. For the sample graph, it can return the path [a, c, g].

Experiment 26: Sum of 1 to N Using Recursion

Aim: To calculate the sum of the first N natural numbers using a recursive function.

Algorithm:

1. Define a function sum(n).
2. Set the base case: if n is 0, return 0.
3. Otherwise, return n + sum(n−1).
4. Call the function with the required positive integer N.
5. Display the returned sum.

Program & Output:

```
def sum_n(n):
    if n == 1:
        return 1
    return n + sum_n(n - 1)

n = 10

print("SUM OF NATURAL NUMBERS")
print("----------------------------")
print("N =", n)

for i in range(1, n + 1):
    print(i, end=" ")

print("\n")

result = sum_n(n)

print("Sum of numbers from 1 to", n, "=", result)
print("Formula: N * (N + 1) / 2")
print("Verification:", n * (n + 1) // 2)
```

```
Python 3.13.1 (tags/v3.13.1:06/1451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
SUM OF NATURAL NUMBERS
----------------------------
N = 10
1 2 3 4 5 6 7 8 9 10

Sum of numbers from 1 to 10 = 55
Formula: N * (N + 1) / 2
Verification: 55
>>>
```

Result: The recursive function calculates the sum correctly. For example, when N = 5, the result is 1 + 2 + 3 + 4 + 5 = 15.

Experiment 27: Vowel Identification

Aim: To identify vowels in a sentence using predefined character-based rules.

Algorithm:

1. Read a sentence from the user.
2. Define the vowels as a, e, i, o and u.
3. Examine each character in the sentence.
4. Convert the character to lowercase for case-insensitive checking.
5. If it matches a vowel, classify it as a vowel; otherwise, ignore it or classify it as a consonant.
6. Display the vowels found and their count.

Program & Output:

```python
def identify_vowels(sentence):
    vowels = "aeiouAEIOU"
    found = []

    print("VOWEL IDENTIFICATION")
    print("---------------------------")
    print("Input Sentence:", sentence)

    for char in sentence:
        if char in vowels:
            found.append(char)
            print("Character:", char, "-> Vowel")

    print("\nVowels Found:", found)
    print("Total Number of Vowels:", len(found))

sentence = "Artificial Intelligence"
identify_vowels(sentence)
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/user/Downloads/AIES.py ==========
========
VOWEL IDENTIFICATION
---------------------------
Input Sentence: Artificial Intelligence
Character: A -> Vowel
Character: i -> Vowel
Character: i -> Vowel
Character: i -> Vowel
Character: a -> Vowel
Character: I -> Vowel
Character: e -> Vowel
Character: i -> Vowel
Character: e -> Vowel
Character: e -> Vowel

Vowels Found: ['A', 'i', 'i', 'i', 'a', 'I', 'e', 'i', 'e', 'e']
Total Number of Vowels: 10
>>>
```

Result: The program identifies vowels in the given sentence regardless of uppercase or lowercase. For example, the vowels in “Artificial Intelligence” are A, i, i, i, a, I, e, i, e and e.